

工具链与工程

host / target / sysroot → 交叉编译 → ELF 分析

Makefile 工程化 → 统一目录规范

— 2.1 / 2.2 / 2.3 —

工程模板：hello + project-template

工具链：riscv64-linux-gnu-gcc

交叉编译全流程：从源码到板端可执行文件

学习目标

host / target / sysroot

区分编译主机、运行设备和目标根文件系统，理解交叉编译

工具链查询

确认交叉编译器目标三元组，定位 sysroot 路径

编译 → 传输 → 运行

使用 hello 工程完成首个交叉编译闭环

ELF 分析

使用 readelf / ldd 查看二进制结构与动态库依赖

架构验证

用 file + readelf 确认产物为 RISC-V ELF

两端验收

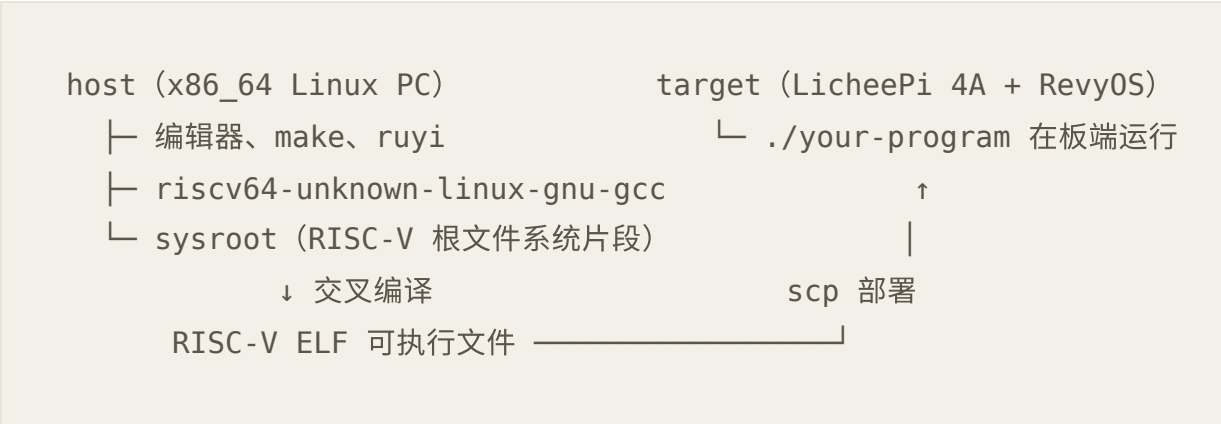
区分 host 编译成功与 target 运行成功两个验收环节

host、target、sysroot 与交叉编译

嵌入式 Linux 开发很少在开发板上直接编译大型程序。更常见的流程是：在性能更好的 PC（host）上生成二进制，再部署到开发板（target）执

行。当 host 与 target 的 CPU 架构不同时，称为**交叉编译**（cross compilation）。

工具链除了编译器，还需要目标系统的头文件和库。这些文件按目标系统的目录结构组织，合称为 **sysroot**。编译时编译器在 sysroot 中查找头文件和库；链接时按目标 ABI 链接其中的库。



环境准备

项目	要求	检查方式
host 架构	记录主机 CPU 架构	<code>uname -m</code>
target 架构	板端为 RISC-V 64 位	<code>ssh user@board-ip</code> <code>uname -m</code>
交叉编译器	前缀 <code>riscv64-unknown-linux-gnu-</code>	<code>command -v riscv64-unknown-linux-gnu-gcc</code>
工程目录	<code>chapters/ch02/code/hello/</code>	<code>ls main.c Makefile</code>

分析工
具

host 有 `readelf`、`file`

`command -v readelf`

关键区分： `riscv64-unknown-elf-gcc` 面向无操作系统环境（裸机）。本课程用户态程序应使用带 `linux-gnu` 的工具链。两者不可互换。

操作步骤

1 确认 host 与 target

主机 `uname -m` (x86_64)，板端 `uname -m` (riscv64) ——两者不同说明需要交叉编译。

2 查看交叉编译器目标三元组与 sysroot

`riscv64-unknown-linux-gnu-gcc -dumpmachine` 应含 `riscv64` 和 `linux-gnu`。`-print-sysroot` 查 sysroot 路径。

3 对比 host 与 target 运行环境

两端分别执行 `file /bin/bash` 和 `ldd /bin/bash | head -n 3`，观察动态链接器路径与架构差异。

4 交叉编译第一个程序

`cd chapters/ch02/code/hello && make clean && make && file hello` — 应显示 RISC-V ELF。

5 部署到板端并运行

`scp hello user@board-ip:~/ && ssh user@board-ip './hello'` — 预期输出 `Hello, RISC-V Linux!`

`readelf -h hello` (Machine=RISC-V)、`readelf -l hello` (LOAD 段)、`ldd hello` (动态库依赖或注明静态链接)。

ELF 关键字段

字段	含义	期望
Class	ELF 位数	ELF64
Data	字节序	小端, LSB
Machine	CPU 架构	RISC-V
Type	文件类型	EXEC 或 DYN

为什么不在主机验证：x86_64 主机无法执行 RISC-V 二进制。./hello 在主机必然报错——正确验收是在板端执行。用 `file` 确认架构，用板端运行确认逻辑。

课堂练习

- 「在 x86_64 笔记本上 `gcc hello.c` 得到的程序，可以直接 `scp` 到 LicheePi 4A 运行。」——这句话是否正确？为什么？
- 若 `scp` 成功但板端提示 `cannot execute binary file`，依次检查哪些项目？
- `readelf -h` 中 `Machine` 显示 `X86-64` 时，构建过程哪里出了问题？

本节成果：host/target/sysroot 对照表 · 交叉编译器三元组与 sysroot 路径 · 板端可运行的 Hello World · 可复用的「编译→scp→运行」命令链 · readelf/ldd 分析记录。

Makefile 与工程目录规范

学习目标

规则与依赖

理解目标、依赖、命令三要素
与 Tab 缩进

变量管理

使用 CC、CFLAGS、
CROSS_COMPILE 避免硬编
码

增量构建

make 时间戳判断 + clean +
.PHONY

目录约定

src/（源码）+ include/（头文
件）+ build/（产物分离）

多文件链接

模式规则将多 .c 编译到 build/
下统一链接

工程复用

模板可扩展至后续 GPIO、
MQTT、综合项目

Makefile 基础

Makefile 描述「什么文件由什么文件生成、执行什么命令」。make 根据文件时间戳决定是否重新编译。

chapters/ch02/code/hello/Makefile 是单文件示例，project-template/Makefile 在此基础上增加多源文件与 build/ 目录。

```
Makefile
CROSS_COMPILE, CC, CFLAGS
↓
hello: main.c
$(CC) $(CFLAGS) -o $@ $^
↓
make / make clean
```

统一工程目录

单文件 hello 适合入门，真实项目应将代码按职责分离：

```
project-template/
├─ Makefile
├─ include/          ← 对外头文件 (greet.h)
├─ src/              ← 源文件 (main.c + greet.c)
└─ build/            ← 产物 (.o + app), 可删可重建
```

编译流程：

```
src/main.c ─┐
src/greet.c ┤─> build/src/*.o ─> build/app
include/ ─┘
```

操作步骤

阶段 A：单文件 Makefile


```
$ cd chapters/ch02/code/hello
$ cat Makefile

# 完整构建流程
$ make clean && make
$ make                # 第二次应提示 up to date
$ touch main.c
$ make                # 应重新编译

# 清理
$ make clean && ls hello 2>&1    # hello 应已删除

# 命令行覆盖变量
$ make CFLAGS='-Wall -Wextra -O0 -g'
```

警告实验：在 `main.c` 中临时引入 `int unused = 0;`，执行 `make` 观察 `-Wall` 警告，修复后恢复原代码。**命令行必须以 Tab 开头——用空格替代会触发 `missing separator` 错误。**

阶段 B：多文件工程

```
$ cd chapters/ch02/code/project-template
$ find . -type f | sort

$ make clean && make
$ file build/app                # RISC-V ELF

# 部署并运行
$ scp build/app user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/app && ./app'
# 预期输出: Hello, RISC-V Linux!
```

```
# 清理重构
```

```
$ make clean && test ! -d build && echo "build removed"
```

工程习惯： build/ 应加入 .gitignore（可删除、可重建）， src/ 和 include/ 纳入版本库。不要把 .o 文件提交到 Git——它们是派生产物，不是源码。

课堂练习

1. 以下 Makefile 错误分别导致什么现象：命令行前用空格而非 Tab；目标未声明依赖且产物已存在； clean 未标记 .PHONY 且目录下有同名文件？
2. 为何 build/ 加入 .gitignore，src/ 和 include/ 纳入版本库？

本节成果： Makefile 变量与规则解释 · make / make clean / 增量构建演示 · 符合 src/include/build 约定的可运行工程 · 板端运行 app 记录 · 可复用模板路径 chapters/ch02/code/project-template/ 。

CoreMark 算力基准

学习目标

基准认知

了解 CoreMark 的测试范围与局限性

板端运行

获取源码、交叉编译、部署到板端并解读分数

理性解读

区分 CPU 算力与真实应用性能（I/O、外设、网络）

本讲为选读，不阻塞主线实验验收。课程综合项目验收不依赖 CoreMark 分数。价值在于建立「这块板子 CPU 大概什么量级」的直觉，为后续是否使用软解码、图像处理等提供参考。

CoreMark 是什么

CoreMark 是 EEMBC 发布的轻量级 CPU 基准测试，主要测试列表操作、矩阵运算、状态机等核心逻辑。结果以 CoreMark 分数或 CoreMark/MHz 报告。它衡量的是 CPU 纯算力——不代表 GPIO 抖动、网络吞吐或存储速度等系统级表现。

操作步骤

1

获取源码

从 EEMBC 官方或课程镜像站获取 CoreMark，解压后阅读 README.md。

2

选择编译方式

板端为 Linux 用户态，应使用 Linux/POSIX 移植（非裸机移植）。

```
export CC=riscv64-unknown-linux-gnu-gcc
make PORT_DIR=linux XCFLAGS="-O2" link
file coremark.exe
```

3

部署与运行

```
scp coremark.exe user@board-ip:~/ && ssh user@board-ip
'./coremark.exe'
```

记录 Iterations、Total time、CoreMark 分数。

4

写简短结论

CoreMark 高分意味着 MQTT 或传感器实验一定更流畅吗？为什么？

理解分数

编译器版本、-O 优化级别、glibc/musl、是否固定 CPU 频率都会导致分数差异。只作同环境前后对比，不追求绝对名次。

课堂练习

列举三项不会被 CoreMark 单独反映、但在本课程实验中很重要的系统能力。

本节成果（可选）：CoreMark 板端运行日志 · 关于「算力基准 vs 课程应用」的简短评述。